

Blatt 6: ActiveRecord-Klassen

Diese Übung sollten Sie in Gruppen von max. vier Studierenden bearbeiten (Mindestgröße ist zwei Personen). Das Ergebnis dieser Übung wird im nächsten Übungsblatt erweitert und führt somit auf die erste Abgabe mit Bonuspunkten hin. Sie haben für beide Übungsblätter zusammen 3 Wochen Zeit. Das Ergebnis von diesem Blatt müssen Sie als Zwischenabgabe Ende nächster Woche abgeben.

Überblick über den Zeitplan:

- Ab 6.11.: Bearbeitung dieser Übung (Blatt 6)
- Ab 13.11.: Bearbeitung der nächsten, auf diesem Blatt aufbauenden Übung 7
- 13.11., 20 Uhr: **Abgabe Zwischenstand** (Ergebnis Blatt 6) per Moodle
- 20.11., 20 Uhr: **Abgabe Endstand** (Ergebnis Blatt 6 und 7) per Moodle
- ab 27.11.: Vorstellung der Lösung in den Übungen (individuell pro Gruppe)

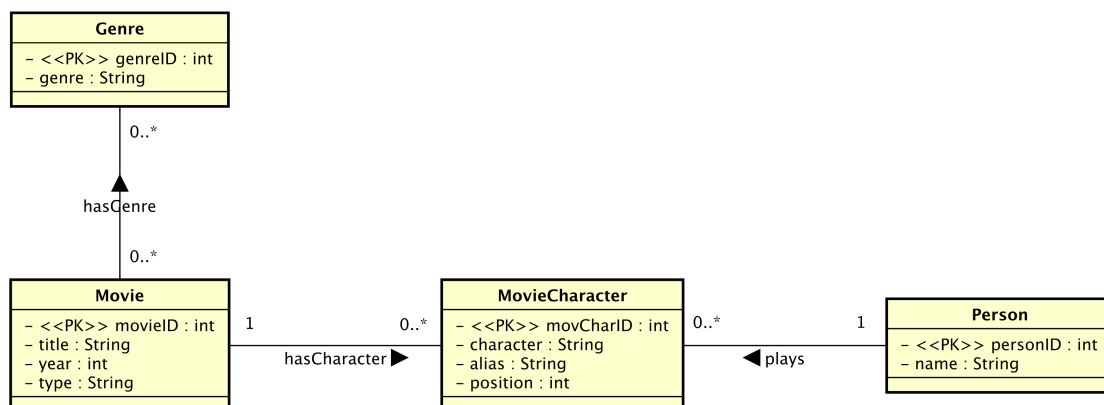
Bitte beachten Sie: Sie müssen den Zwischenstand abgeben, damit die Abgabe des Endstands gewertet wird! Weiterhin müssen Sie mir die fertige Lösung vorstellen und Fragen dazu beantworten können.

Bitte geben Sie Ihr Gruppenergebnis als ZIP-Datei (alle relevanten Java-Quellcode-Dateien und SQL-Skripte) per Moodle ab. Eine entsprechende Abgabemöglichkeit wird rechtzeitig zur Abgabe eingerichtet. Zur Abgabe müssen Sie zunächst ihre Gruppe in Moodle eintragen.

Achtung: Für das Übungsblatt muss relativ viel Code geschrieben werden, da viele (sehr ähnliche) ActiveRecord-Klassen implementiert werden müssen. Daher ist eine sinnvolle Aufteilung der Arbeit in der Gruppe wichtig, um einen vernünftigen Zeitrahmen einzuhalten!

1. Erstellung von Entitätsklassen

Es wird folgendes konzeptionelles Datenbankmodell verwendet (in UML-Notation):



- a) Erstellen Sie ein relationales Datenmodell passend zu diesem konzeptionellen Datenmodell und legen Sie die passenden Tabellen in der Postgres-Datenbank an. Gucken Sie ggf. im Skript von DBS1 nach, wie Sie z.B. die N:M-Beziehung zwischen Movie und Genre anlegen und wie Sie die anderen Beziehungen durch Fremdschlüssel abbilden. Speichern Sie Ihr SQL-Skript zum Anlegen der Tabellen und ergänzen Sie passende DROP-Anweisungen, so dass Sie jederzeit das

Schema neu erzeugen können. Insgesamt müssen fünf Tabellen angelegt werden.

Beachten Sie dabei folgendes:

- Setzen Sie für alle Fremdschlüssel auch einen passenden Constraint (FOREIGN KEY ... REFERENCES ...).
- Setzen Sie für alle Attribute außer Alias und Position einen NOT NULL Constraint.
- Die Join-Tabelle für die N:M-Beziehung zwischen Movie und Genre verwendet einen zusammengesetzten Primärschlüssel aus den beiden Fremdschlüsseln.

b) Erstellen Sie für die Entitäten Genre, Movie, MovieCharacter und Person jeweils eine Java-Klasse mit allen Attributen und passenden get- und set-Methoden. Implementieren Sie zunächst die Methode insert() für alle Klassen. Erstellen Sie ebenso eine Klasse MovieGenre für die Join-Tabelle der Assoziation zwischen Movie und Genre. Erstellen Sie dann auch die update()- und delete()-Methode für Movie.

Beachten Sie dabei folgendes:

- Implementieren Sie ein Verfahren mit Hilfe von Sequenzen zum Erzeugen einer neuen ID in der insert()-Methode – allerdings nur wenn nötig: für MovieGenre ist der Primärschlüssel eine Kombination der Fremdschlüssel.
- Die Fremdschlüssel im Datenmodell werden in den ActiveRecord-Klassen wie normale Attribute behandelt.
- Die Methoden der ActiveRecord-Klassen enthalten keine Transaktionssteuerung (d.h. kein Commit und kein Rollback)!
- Teilen Sie sich die Aufgabe in der Gruppe auf!

c) Testen Sie Ihre Klassen mit folgender Methode. Prüfen Sie anschließend über z.B. pgAdmin, ob die Daten korrekt eingefügt wurden.

```
1 public static void testInsert() throws SQLException {
2     boolean ok = false;
3     try {
4         Person person = new Person();
5         person.setName("Karl Tester");
6         person.insert();
7
8         Movie movie = new Movie();
9         movie.setTitle("Die tolle Komoedie");
10        movie.setYear(2012);
11        movie.setType("C");
12        movie.insert();
13
14        MovieCharacter chr = new MovieCharacter();
15        chr.setMovieId(movie.getId());
16        chr.setPlayerId(person.getId());
17        chr.setCharacter("Hauptrolle");
```

```
18         chr.setAlias(null);
19         chr.setPosition(1);
20         chr.insert();
21
22         Genre genre = new Genre();
23         genre.setGenre("Unklar");
24         genre.insert();
25
26         MovieGenre movieGenre = new MovieGenre();
27         movieGenre.setGenreId(genre.getGenreId());
28         movieGenre.setMovieId(movie.getMovieId());
29         movieGenre.insert();
30
31         DBConnection.getConnection().commit();
32     } catch (Exception e) {
33         DBConnection.getConnection().rollback();
34         throw e;
35     }
36 }
```

d) Implementieren Sie jetzt eine Factory für Movie: MovieFactory mit den beiden Methoden

```
1 public static Movie findById(long id);
2 public static List<Movie> findByTitle(String title);
```

Testen Sie die Methoden, indem Sie ggf. weitere Filme einfügen und nach ihnen suchen.

Aufgabe 1a - Relationales Datenmodell:

Genre(genreID, genre)

hasGenre(genreID, movieID)

ForeignKey(genreID) References Genre(genreID)

ForeignKey(movieID) References Movie(movieID)

Movie(movieID, title, year, type)

MovieCharacter(movCharID, movieID, personID, character, position, alias)

ForeignKey(movieID) References Movie(movieID)

ForeignKey(personID) References Person(personID)

Person(personID, name)

